CSE3063 TERM PROJECT – STUDENT HANDOUT – FALL 2025

1) Project Overview

Build a modular Retrieval-Augmented Generation (RAG) chatbot that answers questions about Computer Engineering department first, then Faculty of Engineering, and finally Marmara University policies and regulations and related topics from local documents (students must download pages and convert them to text possibly manually; no live scraping or API calls! Keep it Simple!). The emphasis is on object-oriented analysis & design quality (GRASP/SOLID), clean interfaces, and traceability from use cases to code (Each main class and method should be traceable to a use case step and shown in the corresponding SSD/DSD diagrams.). No GUI or database is allowed; data must be stored in YAML/JSON files.

For your chatbot RAG dataset, source documents and web sites can be reached from:

- https://www.marmara.edu.tr/universite/yonetim/mevzuat/ (use only Computer Engineering department related content. Please note that some common university wide directives and guidelines still needed to answer questions related to our department such as "M.Ü. Mazeret Sınav Yönergesi")
- https://cse-eng.marmara.edu.tr/
- https://eng.marmara.edu.tr/idari/basili-belgelerformlar

2) Learning Outcomes

- Apply Larman-style OO analysis artifacts at each iteration :
  - Requirement Analysis
    - Requirement Analysis Document (RAD)
    - Domain Model
    - Use Cases
    - System Sequence Diagrams (SSD)
  - Design
    - Design Class Diagram (DCD)
    - Design Sequence Diagrams (DSD).
  - Code
    - Should be in a private GitHub repository named as CSE3063F25GrpX
    - All group members should be included as collaborators in the repository. Also add you instructor (mcganiz) and your TA (_____).
  - Tests
- Collaboration and Project Management
  - Create a private GitHub repository named CSE3063F25GrpX and include all group members as collaborators. Put you group number instead of "X"!
  - Use GitHub Issues to assign and track tasks; each group member should have at least one issue assigned to them.
  - Set up a GitHub Project Kanban board (To Do / In Progress / Done) to manage workflow and demonstrate progress.

    - Ensure a balanced distribution of work among team members, visible through issue assignments and commit history.
    - Your instructor or TA may review issue activity, board updates, and commit logs as part of grading for process and teamwork.
- Design with GRASP (Controller, Creator, Information Expert, Low Coupling/High Cohesion, Polymorphism, Indirection).
- Apply SOLID principles to a modular pipeline with interchangeable strategies.
    - **S**ingle Responsibility Principle – Each class should have only one purpose or reason to change.
    - **O**pen/Closed Principle – Classes should be open for extension but closed for modification.
    - **L**iskov Substitution Principle – Subclasses must be usable wherever their base class is expected.
    - **I**nterface Segregation Principle – Clients should not depend on interfaces they do not use.
    - **D**ependency Inversion Principle – Depend on abstractions rather than concrete implementations.
- Model and implement a RAG pipeline with agents for intent detection, query writing, retrieval, reranking, and answer composition.
- Produce reproducible CLI runs with JSON/YAML configs and JSONL traces.
    - Your program should run from the command line using input settings stored in JSON or YAML files, and every run should generate consistent, repeatable results while recording its detailed execution steps or events as a JSON Lines (JSONL) log file for testing and analysis.

3) Scope & Constraints

- No GUI, no database. All input/output and configuration via CLI + YAML/JSON files.
- Local, ethical corpus only (public university documents).
- Sequential pipeline (no concurrency). Deterministic stubs instead of real LLM/vector DB APIs in the first iteration (For example, instead of using a real embedding model, return fixed numeric vectors for given words.).
- Heavy LLM usage is allowed for drafting prompts and code ideas, but teams must adapt/validate and keep an LLM usage appendix.
- Every and each group member must have 100% command of the code and artifacts. In the demo any group member must be able to answer any question related to the any part of the project.

4) Pipeline Stages & Responsibilities

IntentDetector

- Classify the user question into intents (e.g., Registration, StaffLookup, PolicyFAQ, Course, Unknown).
- Strategy interface with at least one implementation (e.g., RuleIntentDetector).

QueryWriter (+ optional QueryDecomposer)

- Rewrite question into one or more retrieval queries; optionally split complex questions into subqueries.
- At least one heuristic implementation.

Retriever

- Search a local index (KeywordIndex or VectorIndex) and return candidate chunks (hits).
- Strategy interface; choose via config.

Reranker

- Reorder the retrieved hits. Implement at least one reranker (Simple heuristic reranker).
- A second reranker must be added in Iteration 2 (e.g., Hybrid).

AnswerAgent

- Compose final answer from the ranked hits and attach citations (chunk IDs).
- Run schema and citation validation before returning.

RagOrchestrator & Pipeline

- GRASP Controller coordinates the stages with a shared Context object.
- Template Method/Composite to run ordered PipelineStage steps; publish TraceEvents at each step.

Trace/Logger

- Observer-style TraceBus with a JsonlTraceSink that records inputs/outputs/timings/errors for each stage.

Recommended Iteration Plan:

Iteration 1 (Java) — Minimal but Complete Baseline (no LLMs, no vector stores, no BM25)
Goals

- Build a sequential, deterministic RAG-shaped pipeline using only local text files and simple string/keyword logic.
- Emphasize clean interfaces (Strategy, Template Method, Observer) and SOLID.
- No external APIs (LLM/vector DB/Elasticsearch). All behavior must be reproducible from configs.

Data & Preprocessing

- Implement as object-oriented, deterministic components so that indexing can be repeated consistently.
- Source collection: Download a set of department/university PDFs/web pages and convert them to plain text.
- Normalization: lowercasing; optional punctuation removal; keep paragraph/section boundaries if possible.
- Chunking: split into fixed-size windows (e.g., 400–600 characters or ~100–150 words) with overlap (e.g., 50–100 chars). Store chunks with docId, sectionId, startOffset, endOffset in JSON.
- Combined file (optional): also produce combined.txt for simple baselines.
- Index: build a tiny "KeywordIndex" (JSON) mapping token → [list of (docId, chunkId, tf)].

Minimal Components (with simple, testable algorithms)

- IntentDetector (Strategy)
  - Purpose: Decide what the question is about.
  - Interface: IntentDetector.detect(question) → Intent enum {Registration, StaffLookup, PolicyFAQ, Course, Unknown}.
  - Baseline implementation: RuleIntentDetector
  - Rules come from YAML (keywords per intent); match if any keyword occurs (case-insensitive).
  - Ties broken by a deterministic priority list (config).
  - Tests: Given inputs with overlapping keywords, returns the expected intent following the configured priority.
- QueryWriter (Strategy)
  - Purpose: Turn the user question into search terms.
  - Interface: QueryWriter.write(question, intent) → List<String> terms (and optional subqueries).
  - Baseline implementation: HeuristicQueryWriter
  - Lowercase, strip stopwords (from YAML), stem or simple suffix-trim (optional).

    - Keep top N unique terms by order of appearance; if intent == StaffLookup, force add tokens like “staff, advisor, office” from a per-intent term booster list.
    - Tests: Stopword removal, stemming toggle, per-intent boosters.
- Retriever (Strategy)
    - Purpose: Find candidate chunks that contain the query terms.
    - Interface: Retriever.retrieve(terms | subqueries, index) → List<Hit>.
    - Baseline implementation: KeywordRetriever
        - Score each chunk by sum of term frequencies (tf) of matched terms.
        - Return top K (e.g., K=10). If using subqueries, retrieve per subquery then merge (stable merge by score; see deterministic tie-break below).
    - Deterministic tie-break: (score DESC, then docId ASC, then chunkId ASC).
    - Tests: Given a tiny index fixture, verify top-K order and tie-breaks.
- Reranker (Strategy)
    - Purpose: Reorder hits to make the best chunk come first.
    - Interface: Reranker.rerank(query, hits, metadata) → List<Hit>.
    - Baseline implementation: SimpleReranker
        - Scoring formula (deterministic, integer math OK):
        - score = (tf_sum × 10) + proximityBonus + titleBoost where proximityBonus = +5 if any two query terms appear within 15 characters, else 0;
    - titleBoost = +3 if doc title or section header contains any query term. Keep the same stable tie-break rule as Retriever.
    - Tests: Construct small examples to exercise each bonus.
- AnswerAgent
    - Purpose: Produce a short answer and citation(s).
    - Interface: AnswerAgent.answer(query, topHits, store) → Answer {text, citations[]}
    - Baseline implementation: TemplateAnswerAgent
    - • Template: “Your answer: {bestSentence}. See: {docId:sectionId:offsetStart–offsetEnd}”
        - bestSentence: choose the sentence (split on . ! ?) from the top-1 chunk that contains the most query terms; if none, take the first sentence of that chunk.
        - Always attach at least one citation with docId/section/offsets.
    - Tests: Sentence selection, citation formatting, fallback behavior.
- Orchestrator & Pipeline (sequential; Template Method)
    - RagOrchestrator.run(config, question)
        - Steps (Template Method): load config → detect intent → write query → retrieve → rerank → answer → emit trace.
        - Context object carries question, intent, terms, hits, answer.
        - No concurrency/threads.
- Trace/Logger (Observer)

        - TraceBus publishes TraceEvents at each stage with: {stage, inputs, outputsSummary, timingMs, errors?}. JsonlTraceSink writes one JSON object per line (JSONL) in logs/run-YYYYMMDD-HHMMSS.jsonl.
    - CLI (deterministic)

      java -jar rag.jar --config config.yaml --q "Murat Can Ganiz'in ofis numarası nedir?"
    - Sample questions:
        - CSE3063 dersinin ön koşulu nedir?
        - CSE3063 dersi hangi dönemde yer almaktadır?
        - CSE3063 dersinin kredisi kaçtır?
        - Öğrencilere kaç dersten tek ders sınav hakkı tanınır? (This is an actual question recently asked by one of our students :))
    - Outputs
        - stdout: the final answer line (single paragraph).
        - logs/: JSONL trace for each run.
        - citations appear in the answer.

- Acceptance for Iteration 1
    - Deterministic behavior: same input + same config → same output and same top-K ordering.
    - Config-driven strategy selection: swapping to a NoOpReranker (a reranker that does nothing) or changing proximity window in the YAML/JSON config, should change result rankings without code edits.
    - At least two CLI scenarios run end-to-end with valid citations.
    - JSON index and chunk store generated from your corpus; no external search/LLM.
    - Unit tests (≥6 across ≥4 classes) cover: intent rules, stopwords/boosters, retrieval ordering, reranker bonuses, sentence selection, citation formatting.
    - Unit tests must run and pass using a simple local corpus
- UML & Patterns Mapping
    - Strategy: IntentDetector, QueryWriter, Retriever, Reranker.
    - Template Method: Pipeline execution skeleton in Orchestrator.
    - Observer: TraceBus → JsonlTraceSink.
    - Factory Factory (optional): create objects like IntentDetector or Reranker from config file (YAML) entries.
    - SOLID: SRP per stage; DIP via interfaces; OCP by adding new strategies without touching orchestrator.
- What to defer to Iteration 2 (Python)
    - Add a second reranker (e.g., Cosine over hashed n-grams or simple Jaccard) and make it swappable by config.
    - Add a VectorIndex stub only if desired (still deterministic; no external libs).
    - Use a simple vector store (optional)
    - Use an LLM via a free API (optional)
- Batch CLI and basic evaluation metrics JSON.

5) Data & Indexing

- Corpus: at least 10 web pages/pdf files of public university text (policies, registrar, staff directory).
- Chunking: at least 300 chunks (size ~300–600 tokens), store as JSON with doc/section metadata.
- Index: JSON-based KeywordIndex (deterministic).
- Citations: Format as docId:section:span; each answer must include at least one citation.

6) OO Design Principles (Mapping)

- GRASP Controller: RagOrchestrator; Information Expert: Index/ChunkStore; Creator: ComponentFactory.
- SOLID: SRP (one stage = one responsibility); OCP (add strategies via interfaces/factory); DIP (Pipeline depends on abstractions).
- GoF: Strategy (Intent/Retrieval/Rerank), Command (tool-like actions such as index building), Factory (component creation), Adapter (convert between different data sources or formats or embedding/index variants), Template Method (Pipeline), Composite (Plan/steps if you decompose queries), Observer (TraceBus).

7) Required Artifacts (per iteration)

- RAD (≤1 page): vision, goals, glossary, risks, NFRs (traceability, testability).
- Domain Model (UML): core concepts & relationships.
- Use Cases (≥2 fully dressed) (e.g. for different intents…)
- System Sequence Diagrams (SSDs).
- Design Class Diagram (DCD): It should be fully dressed based on UML standards. Show interfaces, abstract classes, dependencies. You may omit simple attributes BUT all reference type attributes must be visible in your diagram so that we can evaluate composition (has-a relationships).
- Design Sequence Diagrams (≥2): one per main use cases or flows (show stage interactions + trace).
- README: commands, config schema, directory layout;
- Unit tests (≥6 across ≥4 classes).

8) Iterations & Milestones

Iteration 1 (3 weeks, Java) — Core pipeline & first strategies

- Implement: RagOrchestrator, Pipeline, Context, IntentDetector (Rule), QueryWriter (Heuristic), Retriever + KeywordIndex, Reranker (BM25 or heuristic), AnswerAgent, TraceBus + JsonlTraceSink, Chunker + JsonChunkStore, IndexBuilder, Schema/CitationValidator.
- Deliverables: Docs PDF (≤7 pages), repo with code/tests, 2 CLI scenarios (registration policy; staff lookup), logs.

Transition Week (1 week) — Java→Python design deltas

- 1–2 pages: what stayed invariant (interfaces/messages), what changed (typing/exceptions/collections).
- At the end of first week, submit Python version of your first iteration Java code that is running without any errors.

Iteration 2 (3 weeks, Python) — Extensibility & evaluation

- Recommendations (specifics to be determined later…): add VectorIndex + EmbeddingProvider (stub), alternate Reranker (Cosine or Hybrid), PolicyRouter, QueryCache, EvalHarness, etc. .
- Batch CLI (`--batch`)
- Evaluation report (coverage@k, simple accuracy, latency).

9) CLI Examples

java -jar rag.jar --config config.yaml --q "Hangi harfli başarı notları başarısız notlardır?"

python rag.py --config config.yaml --batch eval/questions.json

10) LLM Usage Policy

- You may use LLMs for brainstorming, code drafts, doc wording, and test ideas.
- Maintain an Appendix with key prompts, pasted outputs used, and your edits/validations.
- You are responsible for correctness, citations, and design quality—LLM output must be reviewed and adapted.

11) Deliverables Checklist (per iteration)

- Documents:
  1. CSE3063F25_GrpX_Iter1_1_RAD.pdf (including at least two fully dressed use cases.)
  2. CSE3063F25_GrpX_Iter1_2_DomainModel.pdf
  3. CSE3063F25_GrpX_Iter1_3_SSDs.pdf
  4. CSE3063F25_GrpX_Iter1_4_DCD.pdf
  5. CSE3063F25_GrpX_Iter1_5_DSDs.pdf
  6. CSE3063F25_GrpX_Iter1_6_Repo.zip (source + tests + data/index/logs + README, must be less then 10MB)
  7. CSE3063F25_GrpX_Iter1_7_2CLI_output.txt
  8. CSE3063F25_GrpX_Iter1_8_JSONL_traces.jsonl
  9. CSE3063F25_GrpX_Iter1_9_LLM_Appendix.pdf (1 page)

Upload all documents as a single zip file named “CSE3063F25_GrpX_Iter1.zip” to google classroom by the deadline. File size must be less than 20MB.

12) Acceptance Criteria (In 10 minutes Demo)

- All group members must be present in the demo and able to answer design or code questions.
- Two CLI scenarios run successfully and produce answers with valid citations.
- Config swaps reranker without code changes; behavior difference observable.
- UML artifacts align with code (names/contracts).